

Correcting Spelling Mistakes with N-Gram Language Models

Spelling correction is the task of finding an incorrect word and replacing it with the most likely intended word. A useful spell corrector must consider both the spelling of a word and the words surrounding it.

For example:

```
This is a smple sentence
```

smple is not a normal dictionary word, so its spelling can be corrected using nearby words such as **simple**.

Another example is:

```
I had a peace of cake
```

peace is a valid English word, but it is incorrect in this context. The intended phrase is **piece of cake**. This kind of correction requires a language model that understands context.

The implementation in **assignment3.py** uses:

- Unigrams: one-word frequencies.
- Bigrams: frequencies of two consecutive words.
- Trigrams: frequencies of three consecutive words.
- Levenshtein edit distance: similarity between a misspelled word and a candidate.
- Additive smoothing: prevents unseen n-grams from receiving probability zero.
- Log probabilities: makes multiplication of probabilities numerically stable.

1. Types of spelling errors

Non-word errors

A non-word error produces a token that is absent from the learned vocabulary.

```
This is a smple sentence
```

Here, **smple** is not in the vocabulary. The corrector searches for a vocabulary word that is close to **smple** and fits the surrounding context.

Real-word errors

A real-word error produces a valid word, but the word does not fit its context.

```
I had a peace of cake
```

peace may be present in a general dictionary, but the language model should prefer **piece** because **piece of cake** is more likely in the training corpus.

The two error types need different checks:

```
is_non_word = word not in vocabulary
```

- For non-word correction, inspect words that are not in the vocabulary.
- For real-word correction, inspect words that are already in the vocabulary.

2. Install and import the tools

This implementation uses only Python standard-library modules:

```
from collections import Counter
import math
import re
```

Their purposes are:

- **Counter** stores n-gram frequencies.
- **math** provides the natural logarithm.
- **re** extracts word tokens from text.

3. Tokenize the input

Before counting n-grams or correcting words, text must be converted into a consistent sequence of tokens.

```
WORD_PATTERN = re.compile(r"[a-z]+(?:'[a-z]+)?")
```

This pattern matches:

- Lowercase alphabetic words such as **sentence**.
- Apostrophe-containing words such as **don't**.
- It does not retain punctuation as a token.

The tokenizer is implemented as a static method because it does not need access to an object or instance state:

```
@staticmethod
def _tokenize(text):
    return WORD_PATTERN.findall(text.lower())
```

The steps are:

1. Convert the text to lowercase.
2. Find all matches of `WORD_PATTERN`.
3. Return the matches as a list.

Example:

```
_tokenize("I had a Piece of cake!")
# ['i', 'had', 'a', 'piece', 'of', 'cake']
```

Lowercasing ensures that `Piece` and `piece` are treated as the same vocabulary item.

4. Create the spell-corrector class

The complete correction system is grouped into one class:

```
class NGramSpellCorrector:
    """Correct non-word and real-word spelling errors with n-gram scores."""
```

An object of this class stores the language information learned from a training corpus. Once it has been trained, the same object can correct many input sentences.

5. Validate constructor arguments

The constructor receives three configuration values:

```
def __init__(self, corpus, max_edit_distance=2, smoothing=0.1):
```

- `corpus`: sentences used to learn the language model.
- `max_edit_distance`: maximum number of character operations allowed for a candidate.
- `smoothing`: small positive value added to counts.

Validate these values before building the model:

```
if not corpus:
```

```
        raise ValueError("corpus must contain at least one sentence")
    if max_edit_distance < 1:
        raise ValueError("max_edit_distance must be at least 1")
    if smoothing <= 0:
        raise ValueError("smoothing must be greater than zero")
```

This prevents invalid models. An empty corpus cannot provide vocabulary or probabilities, and non-positive smoothing cannot be used in the probability formulas below.

6. Learn unigram, bigram, and trigram counts

Initialize the model state:

```
self.max_edit_distance = max_edit_distance
self.smoothing = smoothing
self.unigrams = Counter()
self.bigrams = Counter()
self.trigrams = Counter()
```

Unigrams

A unigram is one token considered by itself. For this sentence:

```
this is a simple sentence
```

the unigram sequence is:

```
this, is, a, simple, sentence
```

The **Counter** records how often each word occurs:

```
self.unigrams.update(tokens)
```

A frequent unigram gives evidence that a candidate word is generally common in the corpus.

Bigrams

A bigram is an ordered pair of consecutive tokens:

```
(this, is), (is, a), (a, simple), (simple, sentence)
```

Bigrams capture local two-word relationships. For example, **piece of** is likely if it appears frequently in the corpus.

Trigrams

A trigram is an ordered group of three consecutive tokens:

```
(this, is, a), (is, a, simple), (a, simple, sentence)
```

Trigrams provide more context than bigrams. They can distinguish between candidates that have similar individual frequencies.

7. Add sentence boundary tokens

The beginning and end of a sentence have context too. To model them, add special markers:

```
padded_tokens = ["<s>", "<s>"] + tokens + ["</s>"]
```

Two start tokens are used because a trigram needs two previous positions at the start of a sentence.

For:

```
this is a simple sentence
```

the padded sequence is:

```
<s>, <s>, this, is, a, simple, sentence, </s>
```

The bigrams are created with adjacent slices:

```
self.bigrams.update(zip(padded_tokens, padded_tokens[1:]))
```

This produces:

```
(<s>, <s>)  
(<s>, this)  
(this, is)  
(is, a)  
(a, simple)  
(simple, sentence)  
(sentence, </s>)
```

The trigrams are created similarly:

```
self.trigrams.update(
    zip(padded_tokens, padded_tokens[1:], padded_tokens[2:])
)
```

For example, the first trigrams are:

```
(<s>, <s>, this)
(<s>, this, is)
(this, is, a)
```

The constructor repeats these operations for every training sentence:

```
for sentence in corpus:
    tokens = self._tokenize(sentence)
    padded_tokens = ["<s>", "<s>"] + tokens + ["</s>"]
    self.unigrams.update(tokens)
    self.bigrams.update(zip(padded_tokens, padded_tokens[1:]))
    self.trigrams.update(
        zip(padded_tokens, padded_tokens[1:], padded_tokens[2:])
    )
```

Finally, save useful model totals:

```
self.vocabulary = set(self.unigrams)
self.total_words = sum(self.unigrams.values())
```

- **vocabulary** contains all distinct training words.
- **total_words** is the total number of word tokens in the corpus.

8. Generate possible correction candidates

A misspelled word should not be compared with every possible English word from an external dictionary. This implementation uses the words learned from the training corpus as its candidate vocabulary.

```
def _candidates(self, word):
    candidates = [
        (candidate, self._edit_distance(word, candidate))
        for candidate in self.vocabulary
        if abs(len(candidate) - len(word)) <= self.max_edit_distance
    ]
```

```
return [  
    candidate  
    for candidate, distance in candidates  
    if distance <= self.max_edit_distance  
]
```

The length check is a cheap optimization. If two words differ in length by more than the maximum allowed edit distance, they cannot be valid candidates.

For example, with `max_edit_distance=2`, a word at edit distance 1 or 2 may be considered, but a word at distance 3 is discarded.

9. Calculate Levenshtein edit distance

Levenshtein distance is the minimum number of single-character operations needed to transform one string into another.

The allowed operations are:

1. Insertion: add a character.
2. Deletion: remove a character.
3. Substitution: replace one character with another.

For example, the distance between `kitten` and `sitting` is 3:

```
kitten -> sitten (substitute k with s)  
sitten -> sittin (substitute e with i)  
sittin -> sitting (insert g)
```

Dynamic-programming table

For two strings, define a table where each cell stores the smallest edit distance between prefixes of the strings.

The first row represents converting an empty string into prefixes of the second string, so it contains insertion costs:

```
previous_row = list(range(len(second) + 1))
```

For every character in the first string, construct the next row:

```
for first_index, first_character in enumerate(first, start=1):  
    current_row = [first_index]
```

The first value in each new row represents deleting characters from the first string.

For each character in the second string, calculate the three possible operations:

```
insertion = current_row[second_index - 1] + 1
deletion = previous_row[second_index] + 1
substitution = previous_row[second_index - 1] + (
    first_character != second_character
)
```

The substitution cost is **0** when the characters match and **1** otherwise. The smallest operation cost becomes the current cell:

```
current_row.append(min(insertion, deletion, substitution))
```

After processing a row, use it as the previous row for the next character:

```
previous_row = current_row
return previous_row[-1]
```

The implementation keeps only two rows instead of the entire table, reducing memory usage while producing the same final distance.

10. Calculate the n-gram score

A candidate should be judged using three kinds of evidence:

- How common the word is by itself.
- How likely it is after the previous word.
- How likely the three-word context is.

```
def _ngram_score(self, previous_word, word, next_word):
```

First calculate the effective vocabulary size:

```
vocabulary_size = len(self.vocabulary) + 1
```

The extra value accounts for an unseen word when smoothing is applied.

Smoothed unigram probability

```
unigram_probability = (
```

```
        self.unigrams[word] + self.smoothing
    ) / (
        self.total_words + self.smoothing * vocabulary_size
    )
```

Without smoothing, an unseen word would have probability zero. A zero probability would make the whole combined score zero.

Smoothed bigram probability

```
bigram_probability = (
    self.bigrams[(previous_word, word)] + self.smoothing
) / (
    self.unigrams[previous_word] + self.smoothing * vocabulary_size
)
```

This estimates the likelihood of **word** given **previous_word**.

Smoothed trigram probability

```
trigram_probability = (
    self.trigrams[(previous_word, word, next_word)] + self.smoothing
) / (
    self.bigrams[(previous_word, word)]
    + self.smoothing * vocabulary_size
)
```

This estimates the likelihood of the current word and its next word given the previous word.

Use log probabilities

The individual probabilities are combined by adding their logarithms:

```
return (
    math.log(unigram_probability)
    + math.log(bigram_probability)
    + math.log(trigram_probability)
)
```

Normally, independent probabilities would be multiplied:

```
P = P_unigram * P_bigram * P_trigram
```

Using the logarithm changes multiplication into addition:

```
log(P) = log(P_unigram) + log(P_bigram) + log(P_trigram)
```

This is numerically safer because multiplying many small probabilities can underflow toward zero. Since the logarithm is increasing, the candidate with the largest probability also has the largest log score.

11. Correct tokens using context

The main token-correction method is:

```
def _correct_tokens(self, tokens, only_non_words):
```

Make a copy so that the input list is not modified directly:

```
corrected = tokens.copy()
padded_tokens = ["<s>"] + corrected + ["</s>"]
```

The padded list gives each word access to its previous and next context.

Process every original token:

```
for index, word in enumerate(tokens):
    is_non_word = word not in self.vocabulary
```

Select which type of error to process:

```
if only_non_words and not is_non_word:
    continue
if not only_non_words and is_non_word:
    continue
```

This gives two modes:

- `only_non_words=True`: skip known words and correct only unknown words.
- `only_non_words=False`: skip unknown words and correct only known words.

For a word selected for correction, generate candidates:

```
candidates = self._candidates(word)
if not candidates:
    continue
```

If no vocabulary word is close enough, leave the original word unchanged.

Find the context around the current token:

```
previous_word = padded_tokens[index]
next_word = padded_tokens[index + 2]
```

The index offset comes from the single start marker in `padded_tokens`.

Choose the candidate with the maximum n-gram score:

```
best_candidate = max(
    candidates,
    key=lambda candidate: self._ngram_score(
        previous_word, candidate, next_word
    ),
)
```

Do not automatically replace the word. Replace it only when the best candidate scores better than the original:

```
if self._ngram_score(
    previous_word, best_candidate, next_word
) > self._ngram_score(previous_word, word, next_word):
    corrected[index] = best_candidate
    padded_tokens[index + 1] = best_candidate
```

Updating `padded_tokens` is important. If an earlier word is corrected, a later word should be scored using the corrected context.

12. Provide public correction methods

Correct only non-word errors

```
def correct_non_word_errors(self, text):
    tokens = self._tokenize(text)
    return " ".join(
        self._correct_tokens(tokens, only_non_words=True)
    )
```

This method is useful when the goal is to fix words that are absent from the training vocabulary.

Correct only real-word errors

```
def correct_real_word_errors(self, text):
    tokens = self._tokenize(text)
    return " ".join(
        self._correct_tokens(tokens, only_non_words=False)
    )
```

This method considers only known words. It is intended for context errors such as **peace** versus **piece**.

Correct both types

```
def correct_text(self, text):
    tokens = self._tokenize(text)
    tokens = self._correct_tokens(tokens, only_non_words=True)
    return " ".join(
        self._correct_tokens(tokens, only_non_words=False)
    )
```

The order matters:

```
input text
-> tokenize
-> correct non-word errors
-> correct real-word errors
-> join tokens into text
```

Correcting non-word errors first can add known words to the token sequence. Those newly corrected words can then participate in real-word correction.

13. Train and use the corrector

Create a small training corpus:

```
training_corpus = [
    "this is a simple sentence",
    "the student wrote a simple sentence",
    "i had a piece of cake",
    "the piece of cake was delicious",
    "she ate a piece of cake",
]
```

Train the model by constructing the class:

```
corrector = NGramSpellCorrector(training_corpus)
```

The constructor now knows:

- Which words are in the vocabulary.
- How frequently each word occurs.
- Which words commonly appear together.
- Which three-word contexts are common.

Test non-word correction:

```
print(
    corrector.correct_non_word_errors(
        "This is a smple sentence"
    )
)
```

Expected kind of result:

```
this is a simple sentence
```

Test real-word correction:

```
print(
    corrector.correct_real_word_errors(
        "I had a peace of cake"
    )
)
```

The model can prefer **piece** because the training corpus contains the phrase **piece of cake**.

Test both correction stages:

```
print(
    corrector.correct_text(
        "I had a peace of cake and smple tea"
    )
)
```

The exact output depends on the training corpus. A small corpus may not contain enough evidence for every word, so the model should be evaluated with representative data.

14. Full processing workflow

The complete algorithm is:

```
training sentences
  -> lowercase and tokenize
  -> count unigrams, bigrams, and trigrams
  -> build vocabulary

new sentence
  -> lowercase and tokenize
  -> identify unknown words
  -> generate close vocabulary candidates
  -> score candidates with context
  -> replace only if the score improves
  -> repeat for known words with suspicious context
  -> join corrected tokens
```

For a token w_i , the system combines approximate evidence from:

```
word frequency:  $P(w_i)$ 
previous-word context:  $P(w_i \mid w_{(i-1)})$ 
three-word context:  $P(w_i \mid w_{(i-1)}, w_{(i+1)})$ 
```

The best correction is the candidate with the highest combined score, provided it beats the original token.

15. Important limitations and improvements

The model learns only from its corpus

The vocabulary is created from the training sentences, so a correct word that never appears in the corpus cannot be selected. A larger and more representative corpus improves coverage.

Candidate generation can be expensive

The current implementation compares a word with every vocabulary item after the length filter. For a large vocabulary, use an indexed candidate generator, a trie, or a specialized spelling-correction library.

Tokenization removes punctuation

The public methods return tokens joined by spaces. Therefore, punctuation and original formatting are not preserved. A production implementation may keep token positions and restore punctuation after correction.

A small corpus gives weak context scores

The examples work as demonstrations, but real applications need enough text to estimate reliable n-gram counts. Sparse data can make unrelated candidates score similarly.

Real-word correction is risky

A valid word should not be changed merely because another candidate has a slightly higher score. The implementation reduces this risk by replacing the word only when the candidate's score is strictly higher. A real application may also require a confidence margin.

Smoothing is a design choice

A larger smoothing value gives unseen n-grams more probability, while a smaller value relies more heavily on observed counts. The value should be tuned using validation examples.

Character distance does not understand phonetics

Edit distance handles character changes, but not pronunciation-based mistakes such as homophones. A stronger system could combine n-gram scores with phonetic similarity, word frequency, keyboard distance, or a neural language model.

16. Common mistakes

1. Using only edit distance and ignoring the surrounding words.
2. Treating every unknown word as an error when it may be a name or technical term.
3. Correcting real-word errors without a language model.
4. Forgetting sentence boundary markers when calculating n-grams.
5. Using raw probabilities without smoothing, causing zero scores for unseen n-grams.
6. Multiplying many small probabilities directly and risking numerical underflow.
7. Replacing a word without checking whether the candidate scores better than the original.
8. Training on too little text and expecting reliable real-word correction.
9. Using a large maximum edit distance and generating too many unrelated candidates.
10. Assuming the output preserves punctuation when the tokenizer discards it.

Conclusion

An n-gram spell corrector combines character-level similarity with word-level context:

```
text
-> tokenize
-> learn or use vocabulary
-> identify non-word or real-word errors
-> generate edit-distance candidates
```

```
-> calculate smoothed unigram, bigram, and trigram scores
-> choose the highest-scoring improvement
-> return corrected text
```

Edit distance answers:

```
Which vocabulary words look similar to the misspelling?
```

The n-gram language model answers:

```
Which similar word makes the surrounding sentence most likely?
```

Using both signals allows the program to correct unknown misspellings such as **smple** and contextually incorrect known words such as **peace** in **peace of cake**.